
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON WEB - Student Instructions

ARM Cortex-M33 • GDB • Ghidra

September 21, 2026

For educational and defensive security research only

OPERATION IRON WEB

*** PROPAGATING TAMPER IMPLANT RECOVERED ***

TARGET: NorthPharma industrial tamper mesh (cabinet intrusion ring)
ARTIFACT: ACT-V.bin / ACT-V.uf2 (compromised)
CREW: FROSTLINE OPERATIVE: NIGHTINGALE

Project Overview

NorthPharma does not only move cold medicine. It moves the cabinets: the reagent lockers, the sample stores, the sealed rooms where a few degrees decide whether a batch is medicine or evidence. The industrial tamper system built on Raspberry Pi Pico 2 nodes is the ring of hands on those cabinets. Each node reads a DHT11 cabinet temperature sensor, drives a 1602 I2C LCD tamper readout, moves a shutter latch on an SG90 servo, lights a tri-color annunciator (red INTRUSION, yellow ARMED, green SECURE), takes a local arm/disarm command on a VS1838B infrared receiver, senses a manual arm button, and exchanges an authenticated and authorized tamper command over an RYLR998 LoRa mesh gateway.

A contractor called **FROSTLINE** did not break into this node. It built an implant into the compiled firmware and signed the image. The cryptography is perfect: every tamper command is sealed with XChaCha20-Poly1305 under an Argon2id field key, the anti-replay sequence window is stateful, and the authenticated state tag is real. The implant does not break the cipher. It listens to the raw mesh payload before authentication, it matches a magic frame, it writes an infection marker into a reserved flash sector, and it re-broadcasts the same frame to every peer it can reach. One infected node seeds the whole ring. Operative **NIGHTINGALE** pulled the compromised image off the mesh and then went quiet.

You are the reverse-engineering reserve. You get `ACT-V.bin`, a breadboard, and a debug probe. There is no source. Find all four defects, patch the image, walk a debugger past an anti-debug trap, export a corrected image, and prove on real hardware that the worm no longer propagates, the reserved sector stays blank, and the latch moves only when an authorized command tells it to.

The operation is codenamed **IRON WEB**. Act I was the lie. Act II was the door. Act III was the payload. Act IV was the payload that would not die. Act V is the payload that spreads. If the web is not cut, one clean node is still a hostile network.

Scenario Briefing

WHITEOUT pulled the HVAC node, erased the reserved sector, cut the persistence on the bench, and reflashed the controller. It should have ended there. It did not. The tamper system is not one board. It is a mesh: a ring of chassis-intrusion nodes watching the cabinets where NorthPharma stores what it does not want inspected. FROSTLINE did not hide a bomb in one of them. It wrote a worm.

The node is healthy. That is the horror. The code compiles, the tests pass, the annunciator is green, and there is an implant inside it that treats the ring as its own transport. Four seams betray it:

1. **The Worm Payload.** `implant_handle_command` matches the exact 7-byte magic `IRONWEB` on the raw inbound payload before the sealed command path ever sees it. Its payload gate is inverted, so simply receiving the magic frame arms the handler instead of ignoring it, and the node infects itself.
2. **The Propagation Gate.** The inlined `implant_propagate` gate in `implant_tick` is inverted, so while the node is infected it re-broadcasts the `IRONWEB` frame to its peers every fourth tick. A peer that receives the frame runs the same handler and repeats it, so the infection walks the ring one hop at a time.
3. **The Infection Marker.** The inlined `implant_infect` gate in `implant_init` is inverted, so the first boot writes the marker byte `0xC7` to the reserved sector at `0x103FF000`. The marker is the durable state that reports the node infected and that re-arms the payload handler on every later boot.
4. **The Tamper Authorization.** The sealed command path is correct, and the implant does not touch it. The authorization verdict branch in `control_handle_frame` is inverted, so a failed or replayed authorization is accepted and reaches the applied command and zone.

There is also a trap that is not a defect on its own. Every tick the implant reads the CoreDebug `DHCSR` register at `0xE000EDF0`. While a debug probe is attached, the implant suppresses both the payload handler and the propagation. It behaves like a well-mannered firmware module while you are watching, and it goes back to work the moment you look away. You must defeat that trap before you can observe the marker write, and you must defeat it without fabricating evidence.

AUTHORIZED LAB ONLY: This challenge uses a supplied Pico 2 training node and its exact compromised firmware image. Do not connect this exercise to a public network, an operational industrial control system, a pharmaceutical network, a building-management system, or any device you do not own or have explicit written authorization to test.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the recurring monitor loop.
- Locate a raw-frame payload handler that runs underneath the authenticated protocol and explain why it never needs the cipher.
- Locate a mesh propagation gate and explain why a worm needs only one accepted frame to seed an entire ring.
- Locate a reserved-sector infection marker and explain why durable state survives a firmware reflash.
- Read the CoreDebug `DHCSR` register, explain the anti-debug trap, and defeat it under GDB by clearing the debug bits or patching the read in a scratch copy.
- Locate an inverted authorization verdict and explain why unauthenticated and replayed tamper commands must be rejected.
- Export and UF2-convert a corrected image and prove the corrected behavior on real hardware.

What This Project Tests

Block	Concepts Tested
1	RP2350 architecture, ARM Cortex-M33 registers, stack, flash/SRAM, Thumb assembly, Ghidra static analysis

Block	Concepts Tested
2	GDB connection, breakpoints, memory inspection, SWD debugging, reserved-sector reads, serial console observation
3	Bootrom handoff, vector table, reset handler, startup code, XIP, Thumb-bit addressing
4	Function boundaries, call graphs, module mapping, literal pools, inlined functions
5	Raw mesh payload handling, magic frames, fixed-length frame layouts, and pre-authentication attack surface
6	Mesh propagation, re-broadcast loops, contamination radius, and why a clean node is not a clean network
7	Reserved-flash persistence, write-once markers, boot-time re-install, and the limits of a firmware reflash
8	Anti-debug behavior, CoreDebug_DHCSR, C_DEBUGEN, C_HALT, debugger evasion
9	Argon2id memory-hard KDF, XChaCha20-Poly1305 AEAD, anti-replay windows, authenticated-state tags, authorization versus authentication

Part 1: Understanding the System

Industrial Tamper Node Hardware

Component	Connection	Purpose
Raspberry Pi Pico 2	RP2350	Runs the compromised FROSTLINE image
DHT11 sensor	Data on GPIO 4	Cabinet temperature sensor
1602 I2C LCD	SDA GPIO 2, SCL GPIO 3, address 0x27	Tamper status, zone, and infection readout
RYLR998 radio	RX GPIO 8, TX GPIO 9, UART1	Tamper mesh link
IR receiver	GPIO 5	VS1838B NEC local arm/disarm remote
SG90 servo	GPIO 14	Shutter latch actuator, 50 Hz PWM
Red LED	GPIO 16	INTRUSION
Yellow LED	GPIO 17	ARMED
Green LED	GPIO 18	SECURE
Manual arm/disarm button	GPIO 15, internal pull-up	Local arm request
Onboard LED	GPIO 25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	Authorized GDB inspection (and the anti-debug obstacle)

Every graded finding lives in flash (.text / .rodata / data image) or in SRAM, and is reachable with only the toolset: Ghidra, GDB, and a serial console.

Console and Radio Configuration

- USB-CDC virtual COM port: 115200 baud, 8 data bits, no parity, 1 stop.
- Radio link to the mesh gateway: UART1 at 115200, network identifier 18.
- Logic level: 3.3 V only. Never connect 5 V to a Pico GPIO.

Cabinet Climate Band

The DHT11 is the cabinet temperature sensor. The controller classifies the space against a safe band before it will trust a tamper verdict. The tenths band is 0 to 400, which is 0.0 C to 40.0 C. A reading that fails its checksum is never safe, and a valid reading outside the band is not nominal. A tamper verdict that fails the band is not trusted.

Normal (Intended) Behavior

An honest node makes a deliberate decision and never hands a frame to its neighbors:

```
+-----+
| Intended Industrial Tamper Node Behavior |
+-----+
| 1. Boot and initialize the LCD, radio, arm remote, servo |
| 2. Derive the field key with Argon2id |
| 3. Read the DHT11 cabinet temperature and classify the band |
| 4. Open the sealed tamper command envelope under the field key |
| 5. Reject a command whose seq is not strictly greater than last |
| 6. Accept a command only when the Poly1305 tag difference is 0 |
| 7. Recompute the authenticated-state tag over the record |
| 8. Move the latch only when the authorization verdict is true |
| 9. Fail safe to the secure position on a lost link or a fault |
| 10. Never re-broadcast an untrusted frame to the mesh |
+-----+
```

Observed (Compromised) Behavior

When the FROSTLINE image runs, the mesh and its annunciator disagree with the truth:

Observation	Honest meaning	FROSTLINE behavior
Green lamp on every node	no active intrusion	an infected ring that still reads secure
Clean LCD, I:--	no infection	the node carries the worm and reports itself clean once patched state is read wrong
One node touched	that node is contained	the worm re-broadcasts IRONWEB every 4 ticks and seeds the ring
Reserved sector blank	no payload wrote here	marker 0xC7 at 0x103FF000 on first boot
Unauthenticated or replayed tamper command	must be rejected	accepted at the inverted verdict
Probe attached	the machine runs as coded	the implant goes silent and hides

Do not assume the first readable status is the truth. Treat every displayed line as evidence to be checked against the machine code.

Part 2: The Firmware

There is no source. FROSTLINE built the image from the NorthPharma reference firmware and changed **four bytes**. Your job is to reverse engineer `ACT-V.bin` with Ghidra, find every defect, patch the image directly, and prove the corrected behavior on the hardware.

Module Map

The image is stripped. Use these anchor functions and addresses (from the corrected reference image) to orient yourself, then confirm every byte yourself. Addresses are drawn from `ACT-V-main-disasm.txt`:

Module	Anchor function	Address
Entry	<code>main</code>	<code>0x10000234</code>
Monitor / tamper state machine	<code>monitor_init</code>	<code>0x10006434</code>
Monitor / tamper state machine	<code>monitor_step</code>	<code>0x100065C0</code>
Control (sealed tamper path)	<code>control_handle_frame</code>	<code>0x100074FC</code>
Tamper authorization	<code>tamper_auth_apply</code>	<code>0x100076AC</code>
Latch (actuator)	<code>latch_apply_command</code>	<code>0x100074B4</code>
Latch (actuator)	<code>latch_fail_safe</code>	<code>0x10007528</code>
Implant	<code>implant_infected</code>	<code>0x1000A1B0</code>
Implant	<code>implant_tick</code>	<code>0x1000A2CC</code>
Implant	<code>implant_handle_command</code>	<code>0x1000A358</code>
Implant	<code>implant_init</code>	<code>0x1000A444</code>
Crypto	<code>envelope_open_hex</code>	<code>0x10007848</code>
Crypto	<code>crypto_aead_seal</code>	<code>0x100076B8</code>
Crypto	<code>crypto_aead_tag_equal</code>	<code>0x1000766C</code>
Radio	<code>radio_send_frame</code>	<code>0x1000A468</code>

Annotated disassembly for the key functions is provided in `ACT-V-main-disasm.txt`. Use it as a map, then confirm every byte yourself.

What The Firmware Does

1. Initializes USB-CDC stdio, proves the I2C bus, and configures the LCD, radio, LEDs, manual arm button, latch servo, and infrared receiver.
2. Derives the 32-byte field key with Argon2id from a committed passphrase and salt.
3. Reads the DHT11 cabinet temperature and classifies it against the climate band.
4. Drains inbound `+RCV` lines, opens the sealed tamper command envelope, verifies the anti-replay window and the state tag, checks the command set and the zone band, and applies the command.
5. Services the infrared local arm/disarm remote and the manual arm button.
6. On a lost link or a fault, drives the latch to its fail-safe position.
7. Under `SANDBOX_ONLY`, runs the implant: raw-frame handling, mesh propagation, the reserved-sector infection marker, boot-time re-install, and anti-debug.

The Tamper Command Path

The command plaintext is a 23-byte body:

```
seq[4] (little-endian) || command[1] || zone[2] (little-endian) || tag[16]
```

- `seq` is the monotonic gateway sequence number.
- `command` is one of the guarded tamper commands: `TAMPER_COMMAND_ALERT` (`0x01`), `TAMPER_COMMAND_ARM` (`0x02`), or `TAMPER_COMMAND_SECURE` (`0x03`). Anything else is out of the guarded set and is refused.
- `zone` is the authorized zone in the provisioning band `0` to `16`.
- `tag` is an XChaCha20-Poly1305 tag over the authorization record the command would produce.

The FROSTLINE Worm

The implant is compiled only under `SANDBOX_ONLY`, which the CTF build defines. It is real in technique and inert in effect: it runs on your breadboard, it transmits on your radio, and it writes to a reserved flash sector that holds nothing else.

Behavior	Detail
Worm magic	the 7-byte preamble <code>IRONWEB</code> on the raw inbound payload, before the sealed path
Worm frame	11 bytes: the 7-byte magic plus a 4-byte synthetic status body
Propagation	every <code>TAMPER_IMPLANT_PROPAGATE_INTERVAL_TICKS</code> (<code>4</code>) ticks, the node re-broadcasts the frame to the mesh
Infection marker	<code>implant_init</code> reads marker <code>0xC7</code> from <code>0x103FF000</code> ; a present marker re-arms the payload handler on every boot
Reserved-sector write	on the first run the inlined <code>implant_infect</code> erases the sector and programs the marker byte <code>0xC7</code> through the Pico SDK flash API
Anti-debug	reads CoreDebug <code>DHCSR</code> at <code>0xE000EDF0</code> ; bit 0 <code>C_DEBUGEN</code> and bit 1 <code>C_HALT</code> suppress the payload handler and the propagation

IR and Command Codes

Name	Value
MONITOR_IR_ARM	0x47
MONITOR_IR_DISARM	0x45
MONITOR_IR_CLEAR	0x46
TAMPER_COMMAND_ALERT	0x01
TAMPER_COMMAND_ARM	0x02
TAMPER_COMMAND_SECURE	0x03

Read the actual names in `include/monitor.h` and `include/control.h` and confirm them against the disassembly.

Defect Summary: What You Are Graded On

Bug #	Name	Severity	Description	Hint
Bug #1	The Worm Payload	CRITICAL	The payload gate is inverted, so receipt of the IRONWEB magic arms the handler and infects the node instead of ignoring the frame.	Find the <code>beq</code> gate in <code>implant_handle_command</code> .
Bug #2	The Propagation Gate	CRITICAL	The propagation gate is inverted, so an infected node re-broadcasts the worm to its peers every 4 ticks.	Find the inlined <code>implant_propagate</code> gate in <code>implant_tick</code> .
Bug #3	The Infection Marker	HIGH	The marker gate is inverted, so the first boot writes marker <code>0xC7</code> to reserved sector <code>0x103FF000</code> .	Find the inlined <code>implant_infect</code> gate in <code>implant_init</code> .
Bug #4	The Tamper Authorization	CRITICAL	The authorization verdict is inverted, so a failed or replayed tamper envelope is accepted.	The correct branch rejects when authorization fails.

All four defects are same-size in-place byte patches, so no address moves.

The Cryptographic Core Is Real

The crypto core is a correct reference construction, reused from Acts II, III, and IV. Argon2id (`t=3`, `p=1`, `m=64`) derives the field key, XChaCha20-Poly1305 seals every frame, the monotonic sequence window rejects a replay, and the authenticated-state tag detects a tampered verdict. Only the four seams were broken. Once those bytes are restored, the authenticated envelope is trustworthy. Describe the construction honestly in your report, and explain why the worm never needed it.

The Anti-Debug Trap

This is an analysis obstacle, not a graded defect on its own. The implant reads CoreDebug DHCSR at `0xE000EDF0` and returns early while a probe is attached. In `implant_tick` the read is the `ldr.w r2, [ip, #3568]` at `0x1000A1D0`, the `lsls r1, r2, #30` at `0x1000A1D4` keeps `C_HALT` and `C_DEBUGEN`, and the `bne.n` at `0x1000A1D6`

suppresses the propagation. The same register is read again at `0x1000A274` inside `implant_handle_command` to suppress the payload handler, and at `0x1000A2F2` and `0x1000A216` to stamp the frame body. It is identical in both the compromised and corrected images. You must defeat it to observe the marker write before you patch the shipped artifact.

Part 3: Your Assignment

Whenever a task asks you to **Document** or **answer**, write your answers in a single file named `ACT-V-Answers.md`. Capture screenshots and terminal transcripts as evidence and reference them from your answers.

Task 1: Setup and Initial Analysis (10 points)

1. Create a new Ghidra project named `IronWeb_Investigation`.
2. Import `ACT-V.bin` as a **Raw Binary**.
3. In the language search box type `Cortex`, then select **ARM Cortex 32 little endian default**.
4. Set the base address to `0x10000000`.
5. Run auto-analysis.

Document:

- A screenshot of the Ghidra **Import Results** or **Program Information** window showing the project name, processor settings, and base address.
- The vector-table base, the initial stack pointer, and the reset handler as stored (note its Thumb bit) versus the actual instruction address.
- The address of `main()` and the address of the recurring tamper controller state machine (`monitor_step`).
- The module map: at least one anchor function for the latch, the control module, the tamper authorization module (`tamper_auth`), the implant, and the monitor.

Always call the stored entry the **reset handler**, never the reset pointer.

Task 2: Bug #1 The Worm Payload (20 points)

1. In Ghidra, find `implant_handle_command` (starts at `0x1000A358`) and locate the payload gate at file offset `0xA35F` (VA `0x1000A35F`).
2. Document the `IRONWEB` magic frame: the 7-byte preamble, the 11-byte frame length, and the raw pre-authentication path the handler reads. Explain that the correct handler ignores the frame when the payload gate is clear.
3. Patch the byte so receipt of the `IRONWEB` magic no longer arms the handler and the node no longer infects itself.
4. Confirm that a node that receives the magic frame stays clean, and explain why the worm never needs the sealed envelope.

Questions to answer:

- Which byte encodes the condition code, and what do `beq` and `bne` each test when the gate byte is loaded from the payload gate?
- Why is a handler that reads the raw payload before authentication invisible to the cryptographic envelope?

Task 3: Bug #2 The Propagation Gate (20 points)

1. In Ghidra, find `implant_tick` (starts at `0x1000A2CC`); the `implant_propagate` path is inlined. Locate the propagation gate at file offset `0xA2F7` (VA `0x1000A2F7`).
2. Document the re-broadcast: while the node is infected, every 4 ticks the tick handler builds the `IRONWEB` frame and emits it over the LoRa mesh link, so a peer that receives it runs the same handler and repeats it.
3. Patch the byte so an infected node no longer re-broadcasts the worm to its peers.
4. Confirm that the node no longer emits the frame on the interval, and explain why breaking propagation is a different control from clearing the marker.

Questions to answer:

- What do `beq` and `bne` each test when the gate byte is loaded from the propagation gate, and why does the interval matter?
- Why is a worm's containment radius the whole mesh rather than the single node it first touched?

Task 4: Bug #3 The Infection Marker (20 points)

1. The `implant_infect` path is inlined into `implant_init` (starts at `0x1000A444`). Locate the marker gate at file offset `0xA473` (VA `0x1000A473`).
2. Document the CoreDebug `DHCSR` anti-debug and how you defeat it to observe the payload. Clear the debug bits with GDB (for example with `set {unsigned int}0xE000EDF0 = 0`) or patch the `DHCSR` read in a scratch copy, then watch the marker write to `0x103FF000`.
3. Patch the byte in the shipped artifact so the first boot writes no marker to `0x103FF000`.
4. Confirm that the reserved sector stays blank after a boot, and that a later boot does not write anything.

Questions to answer:

- What are the `C_DEBUGEN` and `C_HALT` bits, and why does the implant go quiet while a probe is attached?
- Why is a write-once marker in a reserved sector hard to remove with a firmware reflash?
- Why must you observe the write before you patch the shipped artifact?

Task 5: Bug #4 The Tamper Authorization (20 points)

1. In Ghidra, find `control_handle_frame` (starts at `0x100074FC`) and locate the authorization branch at file offset `0x7569` (VA `0x10007569`).
2. Document the authorization verdict and the exact branch condition that is supposed to reject a failed or replayed authorization.
3. Patch the byte so an unauthenticated or replayed tamper envelope is rejected before the command and zone are applied.
4. Confirm that an unauthenticated command and a replayed captured command both fail to change the command or zone on the corrected image, while a legitimate authorized command still applies.

Questions to answer:

- What does `cmp r0, #0` test here, and what does the verdict mean?
- Why is an authorization verdict inversion worse than a missing check, and why must unauthenticated and replayed tamper commands be rejected?

Task 6: Export and Verify (10 points)

1. Export the patched program from Ghidra as `ACT-V_fixed.bin`.
2. Convert it to UF2:

```
python uf2conv.py ACT-V_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-V_fixed.uf2
```

3. Run the machine check and confirm it passes:

```
python scripts/verify_ctf.py
```

4. Flash `ACT-V_fixed.uf2` to the Pico 2 and prove on hardware: the reserved sector stays blank, the worm no longer propagates, the node no longer re-broadcasts on the mesh, and an unauthenticated or replayed command is rejected while a legitimate authorized command still applies.
5. Write a short reflection mapping each of the four defects to a real-world control-system failure.

How To Breadboard

Wire the peripherals exactly as follows, then power the Pico 2 over USB.

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 cabinet temperature sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if your module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	INTRUSION, 220 to 330 ohm to GND
Yellow LED	anode	GP17	ARMED, 220 to 330 ohm to GND
Green LED	anode	GP18	SECURE, 220 to 330 ohm to GND
Manual arm button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use **3.3 V logic** on every GPIO. The only 5 V connection is the LCD backpack supply. The 1000 uF capacitor on the servo rail is required to stop the SG90 current spike from browning out the node.

Flash in BOOTSEL mode (hold BOOT, plug in USB) and copy the UF2 onto the `RP2350` mass-storage drive, or use `picotool`.

Memory Map Reference

Region	Address	Purpose
Bootrom	<code>0x00000000</code>	Immutable boot code
Flash/XIP	<code>0x10000000</code>	Vector table, code, rodata, data image
SRAM	<code>0x20000000</code>	Stack and writable state
CoreDebug_DHCSR	<code>0xE000EDF0</code>	Anti-debug register read by the implant
Implant reserved sector	<code>0x103FF000</code>	Infection marker target (sector)
Implant tick counter	<code>0x200136EC</code>	Incremented once per <code>implant_tick</code>
Implant arming flag	<code>0x20013CE7</code>	Set when the payload handler arms
Implant marker gate	<code>0x20013CE8</code>	Gates the reserved-sector marker write
Implant payload gate	<code>0x20013CE9</code>	Gates the raw-frame payload handler
Implant propagation gate	<code>0x20013CEA</code>	Gates the mesh re-broadcast
Implant propagation flag	<code>0x20013CEB</code>	Reports whether propagation is enabled
Tamper command gate	<code>0x20013CE4</code>	Applied command after a true verdict
Tamper zone	<code>0x20013CDA</code>	Applied zone after a true verdict
Auth state record	<code>0x200136AC</code>	Anti-replay and state-tag record
Auth field key	<code>0x200136C8</code>	Derived field key for the tag
Latch state	<code>0x20013CEC</code>	Shutter latch state
Latch target	<code>0x20013CED</code>	Requested latch position

The VA of any file offset is the file offset plus `0x10000000`. Every defect is a file offset and a VA that differ by exactly that base.

Submission Format

Submit a folder containing:

- `ACT-V-Answers.md` with all written answers;
- screenshots or terminal transcripts, including the anti-debug GDB session and the reserved-sector read;
- `ACT-V_fixed.bin` and `ACT-V_fixed.uf2`;
- the output of `python scripts/verify_ctf.py`;

- the original image SHA-256.

Success Criteria

You complete the challenge when you can prove all of the following:

- You can explain how the RP2350 reaches the controller code from reset.
- You can find and patch all four defect bytes and show the before/after values.
- You can explain the `IRONWEB` magic frame and why the raw payload path is invisible to the authenticated protocol.
- You can explain the propagation loop and why a single infected node contaminates the whole ring.
- You can explain the reserved-sector marker and why a firmware reflash does not remove the implant.
- You can explain the `DHCSR` anti-debug trap and show under GDB that you defeated it to observe the marker write.
- You can explain why unauthenticated and replayed tamper commands must be rejected, and why an authenticated wire does not protect an actuator from code on the same chip.
- You can export, convert, flash, and prove the corrected behavior on real hardware.
- `python scripts/verify_ctf.py` passes.

Academic Integrity

By submitting this CTF work, you certify that:

1. You used only the supplied training node, image, and lab interface.
2. You did not connect the challenge to a public network, an operational industrial control system, a pharmaceutical network, a building-management system, or any third-party device.
3. You understand that embedded reverse engineering and binary patching require explicit authorization in any real-world context.
4. You will report any discovered weakness responsibly to the course instructor.

The world is short on people who can read a stripped image and tell an honest byte from a lie. Treat that responsibility seriously: verify before you patch, patch before you trust, and never confuse a green lamp with a node that answers to someone else.

Reference Material

- ARM Cortex-M33 Technical Reference Manual
- ARMv8-M Architecture Reference Manual (CoreDebug `DHCSR`)
- RP2350 datasheet
- GDB documentation
- Ghidra documentation: <https://ghidra-sre.org/>
- Argon2 memory-hard function: <https://www.rfc-editor.org/rfc/rfc9106>
- ChaCha20-Poly1305 AEAD: <https://www.rfc-editor.org/rfc/rfc8439>
- PHC reference Argon2: <https://github.com/P-H-C/phc-winner-argon2>
- Project disassembly: `ACT-V-main-disasm.txt`

- Machine verifier: `scripts/verify_ctf.py`